«Код Будущего — программирование на Python для специалистов по информационной безопасности»

Модуль 2. Файлы, данные, простая обработка и визуализация.

ТЕМА Обработка ошибок и исключения

Практическое занятие

Цель:

Научиться писать код без прерываний из-за ошибок.

Задачи:

Разберемся что такое ошибки

Узнаем какие ошибки бывают

Научимся как не прерывать работу программы при появлении ошибки

Научимся обрабатывать ошибки.

Методические указания и инструкции по выполнению задания

1. **Вспомните и повторите теорию, необходимую для выполнения практического задания**

Ключевые моменты:

Ошибки — события, нарушающие выполнение программы. Могут приводить к аварийному завершению или уязвимостям.

Типы ошибок:

Синтаксические (например, опечатка в коде).

Логические (неверный алгоритм).

Исключения (ошибки времени выполнения, например, деление на ноль).

Обработка исключений:

Конструкция try / except:

```
try:
    x = 10 / 0
except ZeroDivisionError:
    print("Деление на ноль!")
```

Блоки else (выполняется при успешном завершении try) и finally (выполняется всегда).

2. **Разберитесь в том, как выполняется задание**

Пример программы с обработкой исключения при делении на ноль.

Задание 1

Простое деление с защитой от нуля:

Запросить у пользователя два числа и вывести результат деления.

Если происходит деление на ноль — вывести сообщение об ошибке.

Пример корректного решения:

```
try:
    a = float(input())
    b = float(input())

    if b == 0:
        print("Ошибка: деление на ноль")
    else:
        print(a / b)

except ValueError:
    print("Ошибка: введите числа")
```

Тренажер: 20201-division-with-zero-protection

Задание 2

Логирование исключений в файл:

Изменить предыдущую программу так, чтобы в случае ошибки сообщение записывалось в лог-файл.

Пример корректного решения:

```
try:
    a = float(input())
    b = float(input())

    if b == 0:
        with open("errors.log", "a", encoding="utf-8")
as f:
            f.write("Ошибка: деление на ноль\n")
    else:
        print(a / b)

except ValueError:
    with open("errors.log", "a", encoding="utf-8") as f:
        f.write("Ошибка: введите числа\n")
```

Тренажер: 20202-Logging-exceptions-to-a-file

Задание 3

Реализовать калькулятор, который:

выполняет сложение, вычитание, умножение, деление

не падает при ошибках (деление на ноль, ввод букв вместо чисел)

записывает ошибки в лог

Описание для тренажера

Напишите программу-калькулятор в файле index.py.

Программа должна строго следовать следующим правилам.

1. Программа считывает три строки через input():
   - первая строка — первое число
   - вторая строка — второе число
   - третья строка — операция (+, -, *, /)

2. Программа выполняет указанную операцию и выводит результат через print без лишних слов. Например, при вводе 5, 3 и + вывод должен быть ровно: 8.0

3. Программа обязана обрабатывать ошибки и не должна завершаться с исключениями.

4. Все ошибки должны записываться в лог-файл calculator.log. Логирование должно быть настроено с уровнем ERROR.

Правила обработки ошибок:

1. Некорректный ввод (например, abc вместо числа):
   - вывести: Ошибка: введите числа
   - в calculator.log должна быть запись, содержащая фразу "введены некорректные данные"

2. Деление на ноль:
   - вывести: Ошибка: деление на ноль
   - в calculator.log должна быть запись, содержащая фразу "деление на ноль"

3. Неизвестная операция (не +, -, *, /):
   - вывести: Неизвестная операция

• в лог-файл писать ничего нельзя

Дополнительные требования:

• stderr должен быть пустым, вся информация выводится только через print.

• При успешной операции лог-файл calculator.log не должен содержать ошибок.

• Лог-файл создаётся только в случае ошибки.

• Название основного файла строго index.py.

• Название лог-файла строго calculator.log.

Программа должна полностью удовлетворять этим требованиям, чтобы корректно пройти автоматические тесты.

Пример корректного решения:

```
import logging
logging.basicConfig(filename='calculator.log', level=logging.ERROR)
def calculator():
    Калькулятор с обработкой ошибок
    try:
        num1 = float(input("Введите первое число: "))
        num2 = float(input("Введите второе число: "))
        operation = input("Выберите операцию (+, -, *, /): ")

        if operation == '+':
            print(f"Результат: {num1 + num2}")
        elif operation == '-':
            print(f"Результат: {num1 - num2}")
        elif operation == '*':
```

```
            print(f"Результат: {num1 * num2}")
        elif operation == '/':
            if num2 == 0:
                raise ZeroDivisionError("Деление на ноль")
            print(f"Результат: {num1 / num2}")
        else:
            print("Неизвестная операция")

    except ValueError:
        logging.error("Ошибка: введены некорректные данные")
        print("Ошибка: введите числа")
    except ZeroDivisionError as e:
        logging.error(f"Ошибка: {e}")
        print("Ошибка: деление на ноль")
    except Exception as e:
        logging.error(f"Неожиданная ошибка: {e}")
        print("Произошла ошибка")
# Пример использования
calculator()
```

Пример корректного решения:

```
import logging

logging.basicConfig(filename='calculator.log',
level=logging.ERROR)

def calculator():
    try:
        num1 = float(input())
        num2 = float(input())
```

```
        operation = input().strip()

        if operation == '+':
            print(num1 + num2)
        elif operation == '-':
            print(num1 - num2)
        elif operation == '*':
            print(num1 * num2)
        elif operation == '/':
            if num2 == 0:
                raise ZeroDivisionError("Деление на
ноль")
            print(num1 / num2)
        else:
            print("Неизвестная операция")

    except ValueError:
        logging.error("Ошибка: введены некорректные
данные")
        print("Ошибка: введите числа")
    except ZeroDivisionError:
        logging.error("Ошибка: деление на ноль")
        print("Ошибка: деление на ноль")
    except Exception as e:
        logging.error(f"Неожиданная ошибка: {e}")
        print("Произошла ошибка")

calculator()
```

Тренажер: 20203-calculator

Задание 4

Чтение файла с защитой от ошибок:

> Написать программу, которая запрашивает у пользователя имя файла и выводит его содержимое построчно.
> Если файл не найден — вывести сообщение об ошибке и записать это событие в лог-файл

Описание для тренажера:

Напишите программу для чтения файла в файле index.py.

Программа должна строго следовать следующим правилам.

1. Программа считывает имя файла через input(), без подсказок и дополнительного текста.
   Имя файла считывается одной строкой.

2. Программа пытается открыть файл с указанным именем и выводит его содержимое построчно.
   Каждая строка файла должна выводиться через print без лишних слов и без добавления лишних символов.
   Концы строк должны обрабатываться корректно: вывод должен совпадать с содержимым файла.

3. Программа обязана обрабатывать ошибки и не должна завершаться с исключениями.

4. Все ошибки должны записываться в лог-файл file_reader.log.
   Логирование должно быть настроено с уровнем ERROR и кодировкой utf-8.

Правила обработки ошибок:

1. Если файл не найден (FileNotFoundError):
   • вывести: Ошибка: файл не найден

• в file_reader.log должна появиться запись, содержащая фразу "файл не найден"

2. Любая другая ошибка:
   • вывести: Произошла ошибка
   • в лог-файл должна быть записана строка с описанием ошибки

Дополнительные требования:

• stderr должен быть пустым. Все сообщения выводятся только через print.
• При успешном чтении файла лог-файл file_reader.log не должен содержать сообщений об ошибках.
• Программа не должна выводить подсказки — только результат или сообщения об ошибках.
• Лог-файл должен создаваться только в случае ошибки.
• Название основного файла строго index.py.
• Название лог-файла строго file_reader.log.

Пример корректного решения:

```
import logging

logging.basicConfig(filename="file_reader.log",
level=logging.ERROR, encoding="utf-8")

def read_file():
    try:
        filename = input().strip()
        with open(filename, "r", encoding="utf-8") as f:
            for line in f:
                print(line.rstrip())
    except FileNotFoundError:
```

```
        logging.error("Ошибка: файл не найден")
        print("Ошибка: файл не найден")
    except Exception as e:
        logging.error(f"Неожиданная ошибка: {e}")
        print("Произошла ошибка")

read_file()
```

Тренажер: 20204-Reading-a-file-with-error-protection

Критерии оценивания

Критерий	Баллы	Требования
1. Корректность работы программы	0-1	– 1 балл: программа работает без ошибок, полностью соответствует условиям задачи, и выводит правильный ответ. – 0 баллов: программа не работает или выдает критичные ошибки, выводит неправильный ответ.
2. Использование необходимых функций	0-2	– 2 балла: при решении использованы все требуемые необходимые языковые средства – 1 балл: при решении использованы не все необходимые языковые средства – 0 баллов: требуемые языковые средства не использованы

Шкала оценивания:

Зачтено — 3 балла

Не зачтено — 0 – 2 балла

Требования к оформлению результата выполнения задания: решение задачи предусматривает ввод программного кода в специальное поле тренажера.